Object Oriented Programming in C

- Motivation
- What is Object-Oriented Programming?
  - Encapsulation
  - Inheritance
  - Polymorphism
- Object-Oriented Programming in C
  - Encapsulation (Classes)
    - Header File
    - Source File
    - Usage Code Example
      - Output
  - Inheritance
    - Header File
    - Source File
    - Usage Code Example
      - Output
  - Polymorphism
    - Model
      - Base
      - Derived
    - Implementation
      - Virtual Function Table (vtable) and Pointer to Virtual Function Table (vptr)
      - Base Class Header File
      - Base Class Source File
      - Derived Class Header File
      - Derived Class Source File
      - Usage Code Example
        - Output
  - Naming Conventions and Idioms
    - Naming
    - Class Infrastructure Functions
    - Class Struct Definitions
    - Base Classes with at least one virtual function
      - Inline Virtual Call Forwarder function
      - Vtable Integration
        - Base Class having virtual functions
        - Derived Class implementing specific virtual functions

Motivation

To facilitate the creation of more and more complex software designs, we have to get used to modularized structures and reusable components.

Working with reusable components gets easier when relying on object-oriented designs (OOD) and programming (OOP).

OOD/OOP enables the creation of complex structures according to well-known design patterns, such as those proposed by the Gang of Four (GoF, see Design Patterns).

What is Object-Oriented Programming?

The following content is based on and partially derived from Object-Oriented Programming by Dr. Miro Samek / Quantum Leaps.

Object-oriented programming (OOP) is a *way of design* based on three fundamental concepts:

- **Encapsulation** – the ability to package data and functions together into classes
- **Inheritance** – the ability to define new classes based on existing classes to obtain reuse and code organization
- **Polymorphism** – the ability to substitute objects of matching interfaces for one another at run-time

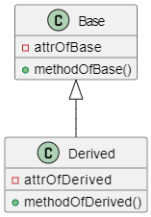
Encapsulation

- ensures the controlled access to methods (member functions) and attributes (member values) of an object, which avoids unexpected manipulations of the inner state of the object
- reveals only information about the "what" (functionality) of a class, but not the "how" (the internal representation)

- defines an interface to the outside and documents it at the same time
- provides significantly improved testability, stability, changeability, and maintainability of the software or its modules

Inheritance

A derived class inherits the functionality and attributes of the base class and can add specialized functionality and attributes in its implementation.



An instance of the `Derived` class on one side provides the `Base` class features:

- `attrOfBase` attribute
- `methodOfBase()` method

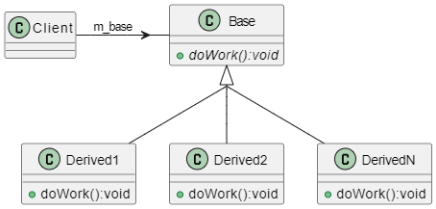
.. and it also provides its own features on the other side:

- `attrOfDerived` attribute
- `methodOfDerived()` method

Polymorphism

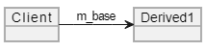
Polymorphism is a concept in object-oriented programming that allows an identifier to take on objects of different data types depending on how it is used. Polymorphism builds directly on inheritance, so the identifier that can take on objects of different data types is a base class type and the objects of different datatypes are all derived from the base class type.

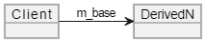
The following class diagram shows a polymorphism example. The client class has a member attribute `m_base` of type `Base`.



The `Base::doWork()` method is virtual, it will be overwritten by the `Derived` classes' method implementations `doWork()` and it does not even need an implementation (this method would then be pure virtual or also known as abstract), thus the *cursive* notation. All the `doWork()` methods need to have the same signature, or in other words, all their parameter lists must be identical and all their return value types must be the same as each other as well.

The `m_base` attribute can hold any object of the type out of the group of the derived classes `Derived1..N`, as shown in the following object diagrams.





The following C++ code snippet presents how the *Client* may use the `m_base` identifier of type `Base` to keep any of the `Derived` typed instances.

```
1 // Client:
```

```cpp
#include "Base.h"
class Client
{
private:
  Base* m_base;

public:
  void work()
  {
    m_base = new Derived1();
    m_base->doWork();
    delete m_base;

    m_base = new Derived2();
    m_base->doWork();
    delete m_base;

    // ..

    m_base = new DerivedN();
    m_base->doWork();
    delete m_base;
  }
};
```

Note: the shown code example is questionable regarding the dynamic memory allocation done on the heap with the quite excessive usage of the new and delete operators, but it's useful to illustrate the polymorphism aspect.

Object-Oriented Programming in C

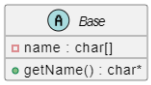
Although the fundamental OOP concepts are traditionally associated with object-oriented languages, such as C++, Java, or C#, you can implement them in almost any programming language including portable, **standard-compliant C** (ISO-C90 Standard).

> C++ was developed to facilitate the implementation of OOP designs.
> However, there may always be reasons why working with C++ is not possible and the programming language C must be used instead.
> To pave the way for the realization of OOP concepts with C, this article will provide the necessary recipes.

Encapsulation (Classes)

In C we can use the **structure** (keyword *struct*) to realize the encapsulation of the member data and functions.

To illustrate how to work with classes using C programming language, the following `Base` class example shall be implemented:



The `Base` class has a name attribute (c-string) which will have to be initialized before using an object of the `Base` class type, and a get accessor member function to retrieve the name string of it.

Header File

```cpp
/* base.h */
#ifndef _BASE_H_
#define _BASE_H_

typedef struct Base Base;
typedef struct BaseAttr
{
  char* name;
} BaseAttr;

struct Base
{
  BaseAttr attr;
  const char* (*getName)(Base const* const me);
};

void Base_init(Base* const me, const char* name);
void Base_finalize(Base* const me);

```

```
20 const char* Base_getName(Base const* const me);
21
22 #endif
```

The core is built up with the `Base` struct, holding the attributes in a nested struct and the `getName()` member **function pointer**. The engagement of an attribute member structure helps to organize the class a little bit and to keep the overview when handling the attribute(s) later on.

Then we have the `Base_init()`, `Base_finalize()`, and the `Base_getName()` function prototypes. The `Base_init()` function has to be called after the allocation of the object before using it, and the `Base_finalize()` function has to be called, when the object will no longer be used, just before destroying it.

Source File

```
1  /* base.c */
2  #include <stdlib.h>
3  #include <string.h>
4  #include "Base.h"
5
6  void Base_init(Base* const me, const char* name)
7  {
8    me->attr.name = malloc(strlen(name)+1);
9    memcpy(me->attr.name, name, strlen(name));
10   me->attr.name[strlen(name)] = '\0';
11
12   me->getName = Base_getName;
13 }
14
15 void Base_finalize(Base* me)
16 {
17   free(me->attr.name);
18 }
19
20 const char* Base_getName(Base const* const me)
21 {
22   return me->attr.name;
23 }
```

The `Base_init()` function allocates memory on the heap for the name attribute storage and assigns the **pointer to the member function** `Base_getName()` to the `getName()` function pointer member. The `Base_finalize()` function will deallocate the `name` attribute memory again when called after the object's lifetime.

The `Base_getName()` function just returns the name attribute in the form of a char pointer as usually done for c-string types.

A key item for making encapsulation work in C is the **me pointer**. The `me` pointer is pointing to the object instance (pointer of own struct type) and has always to be sent whenever calling a function method and we use the convention to make this always the first parameter of any member function. It allows to access the encapsulated content within the object struct.

> The `me` pointer in the C implementation corresponds directly to the implicit `this` pointer in C++.
>
> The `this` identifier is not used, however, because it is a keyword in C++ and such a C program using `this` instead of `me` wouldn't compile when using a C++ compiler.

> Note: in C++ a struct is almost the same as a class, with the only difference being that the default access permission to the struct members is public and the default access permission to class members is private.

Usage Code Example

```
1  #include <stdio.h>
2  #include "Base.h"
3
4  int main()
5  {
6    Base b1, b2;              /* multiple instances of Base */
7
8    Base_init(&b1, "Base1");  /* init b1 instance */
9    Base_init(&b2, "Base2");  /* init b2 instance */
10
11   printf("Base b1 - Name: %s\n", b1.getName(&b1));
12   printf("Base b2 - Name: %s\n", b2.getName(&b2));
13
```

```
  Base_finalize(&b1);
  Base_finalize(&b2);
}
```

Output

```
Base b1 - Name: Base1
Base b2 - Name: Base2
```

Inheritance

We can easily implement single inheritance in C by literally **embedding the inherited class attribute structure** as the **first member** of the derived class structure.

Header File

```c
/* derived.h */

#ifndef _DERIVED_H_
#define _DERIVED_H_

#include "base.h"     /* the base class interface */

typedef struct Derived Derived;
typedef struct DerivedAttr
{
  int derivedVal;
} DerivedAttr;

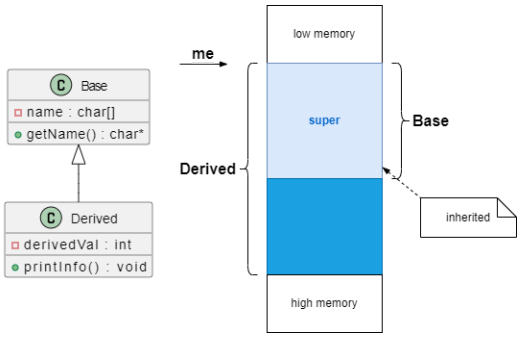
struct Derived
{
  Base super;         /* inherits Base, must be at this struct's first
  DerivedAttr attr;
  void (*printInfo)(Derived const* const me);
};

void Derived_init(Derived* const me, const char* name, int val);
void Derived_finalize(Derived* const me);

void Derived_printInfo(Derived const* const me);
```

With this arrangement, you can always safely pass a pointer to an object of the `Derived` class type to any C function that expects a pointer to an object of the `Base` class type. Specifically, all functions from the `Base` class (also called the superclass) are automatically available to the `Derived` class (called the subclass). So, not only all attributes but also all functions from the superclass are **inherited** by all subclasses (given that the superclass member functions are implemented as function pointers being members of the struct).

The following diagram shows the memory map situation of the `Derived` class type object instance and how the `Base` class as the inherited part is included.



> The alignment of the *Derived* structure and the inherited members from the *Base* structure is guaranteed by the [C Standard WG14/N1124].
> Section 6.7.2.1.13 of this Standard, says: *"... A pointer to a structure object, suitably converted, points to its initial member. There may be unnamed padding within a structure object, but not at its beginning".*

Source File

```c
/* derived.c */
#include <stdlib.h>
#include <stdio.h>
#include "derived.h"
```

```
5
6  void Derived_init(Derived* const me, const char* name, int val)
7  {
8    Base_init(&me->super, name);
9    me->attr.derivedVal = val;
10   me->printInfo = Derived_printInfo;
11 }
12
13 void Derived_finalize(Derived* me)
14 {
15   Base_finalize(&me->super);
16 }
17
18 void Derived_printInfo(Derived const* const me)
19 {
20   printf("Name: %s - Val: %i\n", me->super.getName(&me->super), me->att
21 }
```

The `Derived_init()` function must be called after object creation and just before using it. This function first calls the `Base_init()` function for initializing the inherited part and also passes over the name parameter. As a second step, it assigns the `val` parameter to its own attribute. As the third step, the `Derived_printInfo()` function pointer gets assigned to the structure.

The `Derived_finalize()` function has to be called just before the object's end of life and it calls the `Base_finalize()` function to do the clean-up for the inherited part.

The `Derived_printInfo()` function prints out the attributes of the inherited part as well as the one of its own part.

Usage Code Example

```
1  #include <stdio.h>
2  #include "derived.h"
3
4  int main()
5  {
6    Derived d1, d2;       /* multiple instances of Derived */
7
8    Derived_init(&d1, "Derived1", 1);  /* init d1 instance */
9    Derived_init(&d2, "Derived2", 2);  /* init d2 instance */
10
11   d1.printInfo(&d1);
12   d2.printInfo(&d2);
13
14   Derived_finalize(&d1);
15   Derived_finalize(&d2);
16
17   return 0;
18 }
```

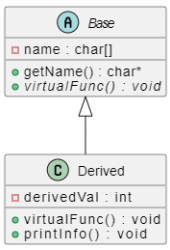
Output

```
1  Name: Derived1 - Val: 1
2  Name: Derived2 - Val: 2
```

Polymorphism

To describe how polymorphism can be implemented with the C programming language, the following model shall be defined.

Model



The following CRC (Class - Responsibilities - Collaborators) cards describe the model in detail.

Base

Responsibilities	Collaborator

• can store a name that may be set during the creation • the name can be get through an accessor • provides an interface as a pure virtual method `virtualFunc()` which has to be implemented by a `Derived` subclass	`Derived` (as a specialization/ specific implementation)

Derived

Responsibilities	Collaborator
• inherits the features of the `Base` class • is a specialization of *Base* class: ◦ can store an integer value (`derivedVal`) that may be set during the initialisation ◦ can print information about the `Base` class' and its own attributes ◦ provides an implementation of the `Base` class' pure virtual method `virtualFunc()` which calls the own `printInfo()` method in the end	`Base` (as a generalization)

Implementation

The source code for the following example can be found at witschi-electronic/lib.emb.oop-c-scheme.

The example depicts how object-oriented programming can be realized with the C programming language.

The `Base` and `Derived` class examples of the previous chapters are re-used and enhanced with the `virtualFunc()` method aspects.

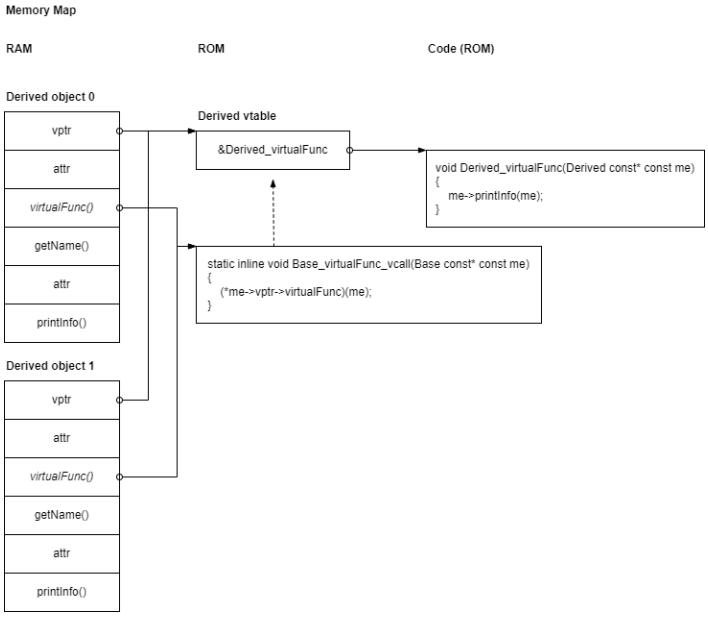
Virtual Function Table (`vtable`) and Pointer to Virtual Function Table (`vptr`)

In addition, the **BaseVtable struct** is introduced here and shows up as a **key concept** that enables polymorphism to be established with implementations using the C programming language.

The `BaseVtable` acts as the virtual function pointer table, which allows the late binding of the virtual function member(s) depending on which `Derived` type will be instantiated (if there are multiple different ones).

The pointer to the virtual function table `vptr` is of type `BaseVtable` and has to be a member of any object derived from `Base` type. Any object of the `Derived` type inherits the `vptr` and the `Derived` 's initialization function has to re-assign the superclass `Base` 's `vptr` with the `Derived` 's concrete virtual function table instance `vtable` .

The following diagram shows the memory map with two `Derived` objects.



Base Class Header File

```c
/* base.h */

#ifndef _BASE_H_
#define _BASE_H_

typedef struct Base Base;
typedef struct BaseVtable BaseVtable;               /* forward declaration of Base's virtual fun
typedef struct BaseAttr
{
    char* name;
} BaseAttr;

struct Base
{
    BaseVtable const* vptr;                         /* Base's pointer to immutable virtual funct
    BaseAttr attr;
    void (*virtualFunc)(Base const* const me);      /* interface function virtFunc() (pure virtu
    const char* (*getName)(Base const* const me);
};

/* Base's virtual function table */
struct BaseVtable
{
    void (*virtualFunc)(Base const* const me);      /* pointer to virtual function virtualFunc()
};

/* Function prototypes */
Base* Base_create(const char* name);
void Base_init(Base* const me, const char* name);
void Base_destroy(Base* const me);
void Base_finalize(Base* const me);

const char* Base_getName(Base const* const me);

/* inline virtual call */
static inline void Base_virtualFunc_vcall(Base const* const me)
{
    (*me->vptr->virtualFunc)(me); /* late binding: calls the function in the current virtual tab
}

#endif
```

Base Class Source File

```c
/* base.c */

#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include "base.h"

```

```c
/* pure virtual function virtFunc(), local (not accessible outside this source file) */
static void Base_virtualFunc(Base const* const me);

Base* Base_create(const char* name)
{
    Base* me = malloc(sizeof(Base));
    Base_init(me, name);
    return me;
}

void Base_init(Base* const me, const char* name)
{
    /* static const (immutable) virtual function table allocation, kept in ROM */
    static const BaseVtable vtable = {
        .virtualFunc = Base_virtualFunc         /* initialize pointer to local pure virtualFunc(
    };
    me->vptr = &vtable;                         /* default assignment to own virtual table */

    me->attr.name = malloc(strlen(name)+1);
    memcpy(me->attr.name, name, strlen(name));
    me->attr.name[strlen(name)] = '\0';

    me->virtualFunc = Base_virtualFunc_vcall;  /* assign I/F with pointer to inline virtual call
    me->getName = Base_getName;
}

void Base_destroy(Base* const me)
{
    Base_finalize(me);
    free(me);
}

void Base_finalize(Base* const me)
{
    free(me->attr.name);
}

/* empty "implementation" of the pure virtual function virtFunc() */
void Base_virtualFunc(Base const* const me)
{
    (void)me;   /* avoid compiler warnings regarding unused parameter */
    assert(0);  /* pure virtual function, shall never be called */
}

const char* Base_getName(Base const* const me)
{
    return me->attr.name;
}
```

Derived Class Header File

```c
/* derived.h */

#ifndef _DERIVED_H_
#define _DERIVED_H_

#include "base.h"     /* the base class interface */

typedef struct Derived Derived;
typedef struct DerivedAttr
{
    int derivedVal;
} DerivedAttr;

struct Derived
{
    Base super;       /* inherits Base, must be at this struct's first position */
    DerivedAttr attr;
    void (*printInfo)(Derived const* const me);
};

Derived* Derived_create(const char* name, int val);
void Derived_init(Derived* const me, const char* name, int val);
void Derived_destroy(Derived* me);
void Derived_finalize(Derived* me);

void Derived_virtualFunc(Derived const* const me);  /* */

void Derived_printInfo(Derived const* const me);

#endif
```

Derived Class Source File

```c
/* derived.c */

#include <stdlib.h>
#include <stdio.h>
#include "derived.h"

Derived* Derived_create(const char* name, int val)
{
    Derived* me = malloc(sizeof(Derived));
    Derived_init(me, name, val);
    return me;
}

void Derived_init(Derived* const me, const char* name, int val)
{
    /* static const (immutable) virtual function table allocation, kept in ROM */
    static const BaseVtable vtable = {
        .virtualFunc = (void (*)(Base const* const me))Derived_virtualFunc  /* init ptr to own v
    };
    Base_init(&me->super, name);
    me->super.vptr = &vtable;                                   /* assign own vtable to Base's vpt

    me->attr.derivedVal = val;

    me->printInfo = Derived_printInfo;
}

void Derived_destroy(Derived* me)
{
    Derived_finalize(me);
    free(me);
}

void Derived_finalize(Derived* me)
{
    Base_finalize(&me->super);
}

/* Derived's specific implementation of virtualFunc() */
void Derived_virtualFunc(Derived const* const me)
{
    me->printInfo(me);
}

void Derived_printInfo(Derived const* const me)
{
    printf("Name: %s - Val: %i\n", me->super.getName(&me->super), me->attr.derivedVal);
}
```

Usage Code Example

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "derived.h"

int main()
{
  uint8_t n = 10;
  Base* b[n];

  /* create 10 objects of Derived type, assign them to an array of pointers to Base type */
  for (int i = 0; i < 10; i++)
  {
    char name[strlen("DerivedN")+1];
    sprintf(name, "Derived%i", i);
    b[i] = (Base*)Derived_create((const char*)name, i);
  }

  /* polymorph call to virtualFunc() on objects of Derived type (through I/F on Base type) */
  for (int i = 0; i < 10; i++)
  {
    b[i]->virtualFunc(b[i]);
  }

  /* clean-up */
  for (int i = 0; i < 10; i++)
  {
    Derived_destroy((Derived*)b[i]);
```

```
  }

  /* trials on a Base object */
  b[0] = Base_create("Base0");
  // b[0]->virtualFunc(b[0]);  /* would abort at run-time*/
  printf("Base - Name: %s\n", b[0]->getName(b[0]));
  Base_destroy(b[0]);

  return 0;
}
```

Output

```
Name: Derived0 - Val: 0
Name: Derived1 - Val: 1
Name: Derived2 - Val: 2
Name: Derived3 - Val: 3
Name: Derived4 - Val: 4
Name: Derived5 - Val: 5
Name: Derived6 - Val: 6
Name: Derived7 - Val: 7
Name: Derived8 - Val: 8
Name: Derived9 - Val: 9
Base - Name: Base0
```

Naming Conventions and Idioms

Naming

Item	Name	Description
Instance variable name of inherited class	`super`	has to be the first member of the core struct of the derived class' struct definition
Object pointer in parameter lists of member functions	`me`	must be the first parameter in a function's parameter list
Attribute instance variable	`attr`	
Pointer to immutable virtual function table	`vptr`	
Instance variable with immutable virtual function table	`vtable`	

Class Infrastructure Functions

Any class shall provide the following infrastructure functions:

Item	Description	Details
`ClassName* ClassName_create()`	constructor function	Allocates memory on the heap for the object and also calls the initializer function. This is a non-mandatory function but is an excellent helper to shape the user code implementation in a bit cleaner way.
`ClassName_init(ClassName* const`	initializer function	Initializes the object, i.e. • assigns parameter values to attributes

		• initializes member function pointers to assign the member function implementations • allocates extra memory for c-string type attributes • for base classes having virtual functions: ◦ creates and initializes default vtable • for derived classes: ◦ initializes superclass object ◦ when the derived class implements specific virtual functions: ▪ creates and initializes specific vtable ▪ re-assigns specific vtable to superclass vptr This has to be called after object creation and before using the object.
`ClassName_destroy(ClassName* me`	destructor function	Calls the finalize function and then de-allocates the memory from the heap for the object. This is a non-mandatory function, but as soon as the create function is implemented, this has to be implemented too. This function has to be called after the usage of the object.
`ClassName_finalize(ClassName* n`	de-initializer function	De-initialize the object, i.e. • de-allocates extra memory for c-string type attributes • for derived classes: ◦ de-initializes the superclass object

Class Struct Definitions

For any class, the following `struct` declarations have to be stated in the header file:

- a forward declaring typedef struct definition for the class to get rid of having always writing struct again and again everywhere

```
typedef struct ClassName ClassName;
```

- for the base classes having at least one virtual function: a forward declaring typedef struct definition for the **Vtable**

```
typedef struct ClassNameVtable ClassNameVtable;
```

- a forward declaring typedef struct definition with the **attribute list**

```
typedef struct ClassNameAttr
{
  /* attributes, i.e:
  int val;
  bool isRunning; */
} ClassNameAttr;
```

- **core** struct definition
  - for base classes having at least one virtual function

```
struct ClassName
{
  ClassNameVtable const* vptr;  /* pointer to immutable virtual function table */
  ClassNameAttr attr;           /* instance of attribute list */
  /* list of function pointers to virtual functions, i.e:
```

```
  void (*virtualFunc)(ClassName const* const me); */
  /* list of function pointers for member functions, i.e:
  const char* (*getName)(Base const* const me); */
};
```

- for derived classes

```
struct ClassName
{
  ClassNameBase super;  /* inherits Base, must be at this struct's first position */
  ClassNameAttr attr;   /* instance of attribute list */
  /* list of function pointers for member functions, i.e:
  void (*printInfo)(ClassName const* const me); */
};
```

- for normal classes

```
struct ClassName
{
  ClassNameAttr attr;   /* instance of attribute list */
  /* list of function pointers for member functions, i.e:
  const char* (*getName)(Base const* const me); */
};
```

- for base classes having at least one virtual function: the **virtual function table** definition

```
struct ClassNameVtable
{
  /* pointer(s) to the virtual function(s) */
  void (*virtualFunc)(ClassName const* const me);
};

```

Base Classes with at least one virtual function

Any base class having virtual functions (functions that are intended to be overwritten by derived classes) shall provide the following implementations.

Inline Virtual Call Forwarder function

In the header file, a static inline vcall function implementation for each virtual function shall be defined as follows:

```
/* inline virtual call for virtualFunc() */
static inline void ClassName_virtualFunc_vcall(ClassName const* const me
{
  /* late binding: calls the function in the current virtual table */
  (*me->vptr->virtualFunc)(me);
}
```

Vtable Integration

In the source file, within the `init()` function implementation:

Base Class having virtual functions

```
void ClassName_init(ClassName* const me)
{
  /* static const (immutable) virtual function table allocation, kept i
  static const ClassNameVtable vtable = {
    &ClassName_virtualFunc    /* initialize pointer to local pure virtu
  };
  me->vptr = &vtable;         /* default assignment to own virtual tabl

                              /* assign I/F with pointer to inline virt
  me->virtualFunc = &ClassName_virtualFunc_vcall;

  /* further implementations */
}
```

Derived Class implementing specific virtual functions

```
void ClassName_init(ClassName* const me)
{
  /* static const (immutable) virtual function table allocation, kept i
  static const ClassNameBaseVtable vtable = {
    /* init ptr to own virtualFunc() impl. with "me" upcast */
    (void (*)(ClassNameBase const* const me))&ClassName_virtualFunc
  };
  ClassNameBase_init(&me->super, name);
  me->super.vptr = &vtable;   /* assign own vtable to Base's vptr (over

```

```
  /* further implementations */
}
```